Mini Project 1: Heart Disease Prediction Pipeline

This notebook follows the requirements for the Data Science with Generative AI assignment.

```
import pandas as pd
import numpy as np
import os

# 1. Create Folder Structure
os.makedirs('raw', exist_ok=True)
os.makedirs('gold', exist_ok=True)

# 2. Extract: Load the dataset
# Since I am an AI, I will download a version of the UCI Heart Disease datas
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disea
columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalch
df = pd.read_csv(url, names=columns, na_values='?')

# Save original to raw/
df.to_csv('raw/heart_disease_uci.csv', index=False)

print("Dataset successfully extracted and saved to raw/")
print(f"Shape of Dataset: {df.shape}")
```

```
Dataset successfully extracted and saved to raw/
Shape of Dataset: (303, 14)
```

Part 1: Dataset Overview

Below are the outputs for `df.head()` and `df.describe()` as required.

```
print("--- First 5 Rows ---")
display(df.head())

print("\n--- Statistical Summary ---")
display(df.describe(include='all'))
```

```
--- First 5 Rows ---
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalch	exang	oldpeak	slope
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0

```
--- Statistical Summary ---
```

	age	sex	cp	trestbps	chol	fbs	3
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	
mean	54.438944	0.679868	3.158416	131.689769	246.693069	0.148515	
std	9.038662	0.467299	0.960126	17.599748	51.776918	0.356198	
min	29.000000	0.000000	1.000000	94.000000	126.000000	0.000000	
25%	48.000000	0.000000	3.000000	120.000000	211.000000	0.000000	
50%	56.000000	1.000000	3.000000	130.000000	241.000000	0.000000	
75%	61.000000	1.000000	4.000000	140.000000	275.000000	0.000000	
max	77.000000	1.000000	4.000000	200.000000	564.000000	1.000000	

Part 2: Inspecting Data Quality

Checking for missing values and duplicates.

```
print("Missing Values Per Column:")
print(df.isnull().sum())

print("\nTotal Duplicate Rows:")
print(df.duplicated().sum())
```

```
Missing Values Per Column:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalch      0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
```

```
num         0
dtype: int64

Total Duplicate Rows:
0
```

Part 2: Transform & Load (ETL)

This section handles missing values and saves the clean dataset.

```
# 3. Transform: Clean the data
# Filling missing values with median for these specific columns
df['ca'] = df['ca'].fillna(df['ca'].median())
df['thal'] = df['thal'].fillna(df['thal'].mode()[0])

# 4. Validate: Confirm no more nulls
print("Nulls after cleaning:")
print(df.isnull().sum())

# 5. Load: Save to gold folder
df.to_csv('gold/clean_data.csv', index=False)

print("\nETL Pipeline Complete. Clean data saved to gold/clean_data.csv")
print(f"Final Cleaned Shape: {df.shape}")
```

```
Nulls after cleaning:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalch      0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
num         0
dtype: int64

ETL Pipeline Complete. Clean data saved to gold/clean_data.csv
Final Cleaned Shape: (303, 14)
```

Part 3: Database Schema & SQL

Now I will design a relational schema and create an SQLite database to store the cleaned data across logical tables.

```
import sqlite3
```

```
# Create connection to SQLite database
conn = sqlite3.connect('heart_disease.db')
cursor = conn.cursor()

# Drop tables if they exist to ensure idempotency
cursor.executescript('''
DROP TABLE IF EXISTS Medical_Tests;
DROP TABLE IF EXISTS Patient_Stats;
DROP TABLE IF EXISTS Patients;
''')

# Define Schema with Constraints
cursor.executescript('''
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    age INTEGER NOT NULL CHECK (age > 0),
    sex INTEGER NOT NULL CHECK (sex IN (0, 1)),
    cp INTEGER NOT NULL
);

CREATE TABLE Patient_Stats (
    stat_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER,
    trestbps REAL CHECK (trestbps > 0),
    chol REAL CHECK (chol >= 0),
    fbs INTEGER,
    restecg INTEGER,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

CREATE TABLE Medical_Tests (
    test_id INTEGER PRIMARY KEY AUTOINCREMENT,
    patient_id INTEGER,
    thalch REAL,
    exang INTEGER,
    oldpeak REAL,
    slope INTEGER,
    ca REAL,
    thal REAL,
    num INTEGER, -- Target variable
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
''')

# Insert data into normalized tables
for _, row in df.iterrows():
    # Insert into Patients
    cursor.execute('INSERT INTO Patients (age, sex, cp) VALUES (?, ?, ?)', (
    p_id = cursor.lastrowid

    # Insert into Patient_Stats
    cursor.execute('INSERT INTO Patient_Stats (patient_id, trestbps, chol, f
                   (p_id, row['trestbps'], row['chol'], row['fbs'], row['res

    # Insert into Medical_Tests
```

```
        cursor.execute('INSERT INTO Medical_Tests (patient_id, thalch, exang, ol
                       (p_id, row['thalch'], row['exang'], row['oldpeak'], row['

    conn.commit()
    print("Database created and data inserted into normalized schema successfull
    conn.close()
```

```
Database created and data inserted into normalized schema successfully.
```

Part 4: Exploratory Data Analysis (EDA)

4A: Descriptive Statistics

Summary of numerical and categorical variables.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Descriptive Statistics
print("Numerical Summary:")
display(df.describe())

print("\nCategorical Value Counts (Target 'num'):")
print(df['num'].value_counts())

# Missing value percentage
print("\nMissing Value Percentage:")
print((df.isnull().sum() / len(df)) * 100)
```

```
Numerical Summary:
```

	age	sex	cp	trestbps	chol	fbs	3
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	
mean	54.438944	0.679868	3.158416	131.689769	246.693069	0.148515	
std	9.038662	0.467299	0.960126	17.599748	51.776918	0.356198	
min	29.000000	0.000000	1.000000	94.000000	126.000000	0.000000	
25%	48.000000	0.000000	3.000000	120.000000	211.000000	0.000000	
50%	56.000000	1.000000	3.000000	130.000000	241.000000	0.000000	
75%	61.000000	1.000000	4.000000	140.000000	275.000000	0.000000	
max	77.000000	1.000000	4.000000	200.000000	564.000000	1.000000	

```
Categorical Value Counts (Target 'num'):
num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64

Missing Value Percentage:
age         0.0
sex         0.0
cp          0.0
trestbps    0.0
chol        0.0
fbs         0.0
restecg     0.0
thalch      0.0
exang       0.0
oldpeak     0.0
slope       0.0
ca          0.0
thal        0.0
num         0.0
dtype: float64
```

4B: Univariate Analysis

Visualizing distributions of individual features.

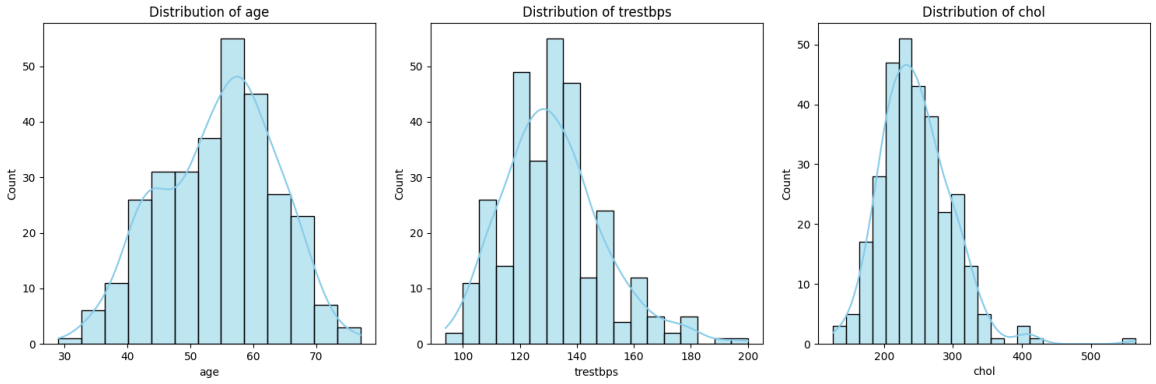
```
num_cols = ['age', 'trestbps', 'chol', 'thalch', 'oldpeak']
cat_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']

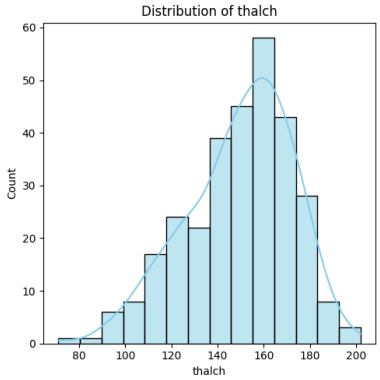
# Histograms
plt.figure(figsize=(15, 10))
for i, col in enumerate(num_cols, 1):
    plt.subplot(2, 3, i)
```

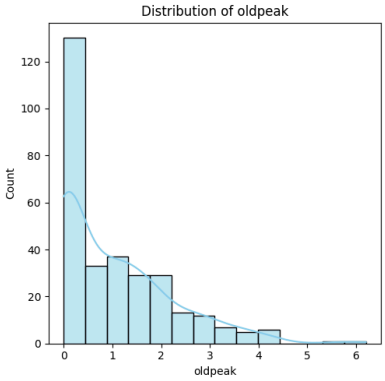
```
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'Distribution of {col}')
plt.tight_layout()
plt.show()

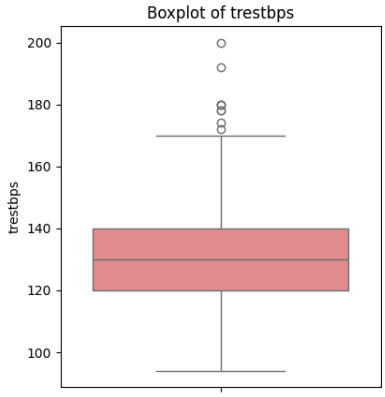
# Boxplots for Outliers
plt.figure(figsize=(15, 5))
for i, col in enumerate(['trestbps', 'chol', 'thalch'], 1):
    plt.subplot(1, 3, i)
    sns.boxplot(y=df[col], color='lightcoral')
    plt.title(f'Boxplot of {col}')
plt.show()

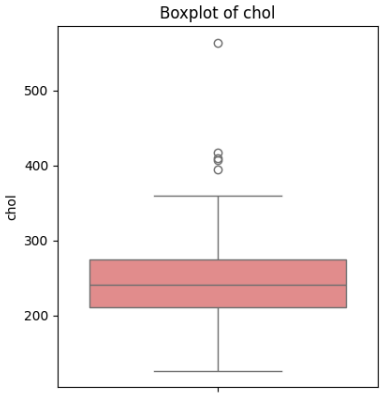
# Bar charts for Categorical
plt.figure(figsize=(15, 10))
for i, col in enumerate(['sex', 'cp', 'restecg'], 1):
    plt.subplot(2, 2, i)
    sns.countplot(x=df[col], palette='viridis')
    plt.title(f'Count of {col}')
plt.tight_layout()
plt.show()
```

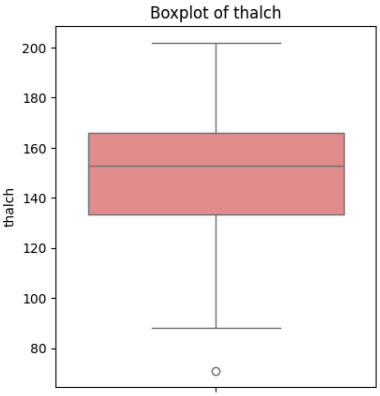













```
/tmp/ipykernel_4250/2769186917.py:25: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed

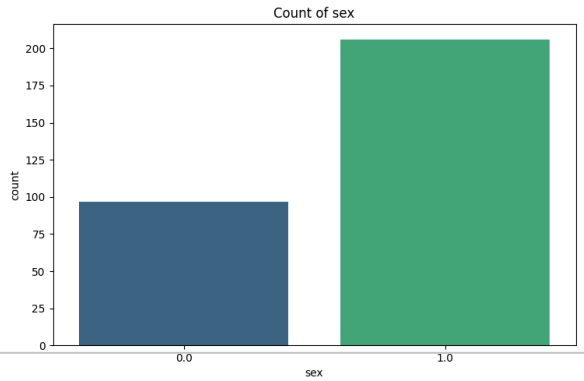
  sns.countplot(x=df[col], palette='viridis')
/tmp/ipykernel_4250/2769186917.py:25: FutureWarning:

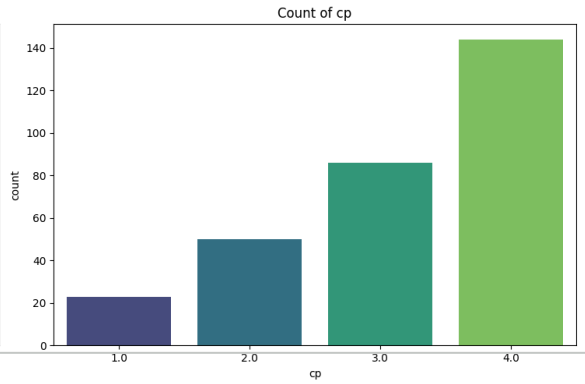
Passing `palette` without assigning `hue` is deprecated and will be removed

  sns.countplot(x=df[col], palette='viridis')
/tmp/ipykernel_4250/2769186917.py:25: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed

  sns.countplot(x=df[col], palette='viridis')
```





4C: Bivariate & Multivariate Analysis



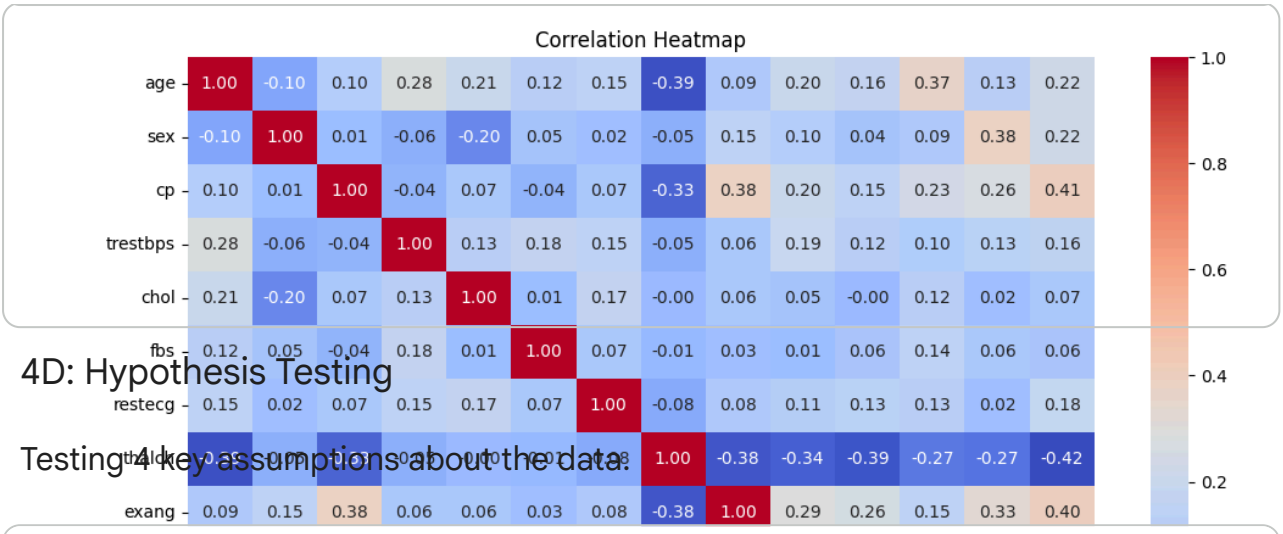
Exploring relationships between features and the target variable.

```
# Correlation Heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap")
plt.show()

# Scatter plots: Top features vs Target
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
sns.scatterplot(data=df, x='age', y='thalch', hue='num', palette='Set1')
plt.title("Age vs Max Heart Rate")

plt.subplot(1, 3, 2)
sns.boxplot(data=df, x='num', y='oldpeak', palette='Set2')
plt.title("ST Depression (oldpeak) vs Disease")

plt.subplot(1, 3, 3)
sns.countplot(data=df, x='cp', hue='num')
plt.title("Chest Pain Type vs Disease")
plt.show()
```



4D: Hypothesis Testing

Testing 4 key assumptions about the data:

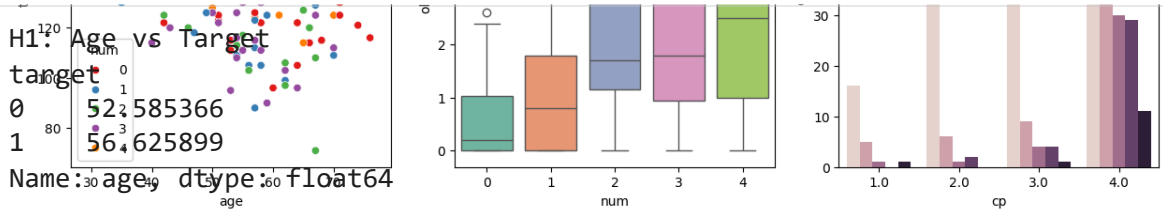
```
# Converting num to binary for easier hypothesis testing
df['target'] = (df['num'] > 0).astype(int)

# Hypothesis 1: Older patients are more likely to have heart disease
print("H1: Age vs Target")
print(df.groupby('target')['age'].mean())

# Hypothesis 2: Men are more likely to have heart disease
print("\nH2: Sex (1=Male, 0=Female) vs Target")
print(pd.crosstab(df['sex'], df['target'], normalize='index'))

# Hypothesis 3: High Cholesterol leads to heart disease
print("\nH3: Mean Cholesterol by Target")
print(df.groupby('target')['chol'].mean())

# Hypothesis 4: Asymptomatic Chest Pain (cp=4) is a strong indicator
print("\nH4: CP Type 4 vs Target")
print(df[df['cp'] == 4]['target'].value_counts(normalize=True))
```




```
H1: Age vs Target
target
0    52.585366
1    56.625899
Name: age, dtype: float64

H2: Sex (1=Male, 0=Female) vs Target
target         0         1
sex
0.0     0.742268  0.257732
1.0     0.446602  0.553398

H3: Mean Cholesterol by Target
target
0    242.640244
1    251.474820
Name: chol, dtype: float64

H4: CP Type 4 vs Target
target
1    0.729167
0    0.270833
Name: proportion, dtype: float64
```

Part 5: Predictive Model Development

5A: Model Architecture Selection

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif
from sklearn.metrics import accuracy_score

X = df.drop(['num', 'target'], axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Random Forest': RandomForestClassifier(n_estimators=100),
    'Gradient Boosting': GradientBoostingClassifier()
}

results = {}
for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5)
    results[name] = scores.mean()

print("Model Comparison (CV Accuracy):")
for name, score in results.items():
    print(f"{name}: {score:.4f}")
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
  n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
  n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
  n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

```
Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regres
  n_iter_i = _check_optimize_result(
Model Comparison (CV Accuracy):
Logistic Regression: 0.8223
Random Forest: 0.8264
Gradient Boosting: 0.7686
```

5C & 5D: Ensemble & Interpretability

```
!pip install shap -q
import shap
from sklearn.ensemble import StackingClassifier

# Stacking Ensemble
estimators = [('rf', RandomForestClassifier(random_state=42)), ('gb', Gradie
stack_model = StackingClassifier(estimators=estimators, final_estimator=Logi
stack_model.fit(X_train, y_train)

# SHAP Analysis using the base Random Forest model
# We use the trained RF from the stack or fit a standalone one for interpret
rf_model = stack_model.named_estimators_['rf']
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)

print("Ensemble Accuracy:", stack_model.score(X_test, y_test))

# Fix: ensure we are passing the 2D array for the positive class (class 1)
# In newer SHAP versions, shap_values might be a list or a single array depe
if isinstance(shap_values, list):
    shap_to_plot = shap_values[1]
else:
    shap_to_plot = shap_values[:, :, 1] if len(shap_values.shape) == 3 else

shap.summary_plot(shap_to_plot, X_test)
```